Übung 8 - TypeScript 1

Erstellen Sie auf Ihrem Computer einen neuen Ordner für diese Übung, in welchem Sie die Lösungen der nachfolgenden Aufgaben ablegen, z. B. `Studium\SoSe25\Praktische Informatik 1\Übung 8`.

Bearbeiten Sie die Übungen mit WebStorm und legen Sie die Lösungen für die folgenden Aufgaben in einer oder mehreren TypeScript-Dateien ab.

Aufgabe 1 (**Node.js und WebStorm**)

In dieser Übung sollen Sie Ihre Entwicklungsumgebung für TypeScript mit Node.js, npm und WebStorm einrichten und ein erstes TypeScript-Projekt erstellen.

Bitte nutzen Sie die in Moodle bereitgestellten **Folien** als Leitfaden. Diese enthalten alle notwendigen Informationen, um die Aufgabe erfolgreich zu bearbeiten.

(a) Laden Sie die aktuelle Version von Node.js von der offiziellen Webseite herunter und installieren Sie diese: https://nodejs.org.

(b) Überprüfen Sie nach der Installation in der Konsole (cmd, Powershell, Terminal), ob die neuesten Versionen von Node.js und npm installiert ist. Nutzen Sie dafür die Befehle `node -v` und `npm -v`.

(c) Laden Sie die IDE **WebStorm** von der offiziellen JetBrains-Webseite herunter.

(d) Installieren Sie WebStorm gemäß den Anweisungen auf der Webseite und den bereitgestellten Folien.

(e) Starten Sie die IDE und loggen Sie sich mit Ihrer THM-Kennung ein, um die kostenlose Studentenlizenz zu nutzen.

(f) Überprüfen Sie Ihre WebStorm-Einstellungen (TypeScript-Support) gem. den Vorgaben der bereitgestellten Folien.

Aufgabe 2 (**Ein erstes TypeScript-Projekt in WebStorm erstellen**)

(a) Öffnen Sie WebStorm und erstellen Sie ein neues leeres Projekt.

(b) Erstellen Sie eine neue Datei mit dem Namen `index.ts`.

(c) Schreiben Sie folgenden TypeScript-Code in die Datei:

```
1 function greet(): void {
2     console.log("Hello, TypeScript!");
3 }
4 greet();
```

(d) Überprüfen Sie, ob automatisch eine Javascript-Datei generiert wurde: `index.js`

(e) Führen Sie die generierte `index.js`-Datei mit Node.js aus - Entweder über das Kontextmenü oder in der Console: `node index.js` und überprüfen Sie, ob die Ausgabe `"Hello, TypeScript!"` in der Konsole erscheint.

Aufgabe 3 (**TypeScript-Aufgaben von leicht nach schwer**)

(a) Zählen Sie die Anzahl `true`-Werte in einem Boolean-Array.

(b) Nutzen Sie ein Enum `Color { Red, Green, Blue }` und verwenden Sie `switch`, um je Farbe einen Text Ihrer Wahl auszugeben. Beispiel: "Red is warm".

(c) Zählen Sie, wie oft ein bestimmtes Zeichen in einem String vorkommt. Beispiel: In "banana"kommt 'a' dreimal vor.

(d) Ergänzen Sie den folgenden Code:

```
const person: [string, number] = ["Alice", 25];
const name = person[0];
const age = person[1];
const sentence = `... is ... years old`;

console.log(sentence);
```

(e) Deklarieren Sie eine Variable, sodass ihre Werte Zahlen oder Strings sind, und weisen Sie einen Wert zu. Ermitteln Sie dann, ob es eine Zahl oder ein String ist. Tipp: `typeof`.

(f) Berechnen Sie die Fibonacci-Funktion `fib` für die ersten 10 natürlichen Zahlen mittels einer Schleife und geben Sie die Zahlen auf der Konsole aus. Erinnerung: $fib(1) = fib(2) = 1, fib(n) = fib(n-1) + fib(n-2)$.

(g) Ändern Sie Ihr Programm so, dass es die Zahlen nicht einzeln ausgibt, sondern sie in einem Array speichert und geben Sie am Ende das Array auf der Konsole aus.

(h) Benutzen Sie eine for-Schleife, um eine Liste umzudrehen. Beispiel: liste1 = [1,5,3,2]. Das Umdrehen ergibt: liste2 = [2,3,5,1]; Als Eingabeliste können Sie die in der vorigen Aufgabe erzeugte Liste verwenden. Geben Sie die Liste mittels einer for-of Schleife aus.

(i) Schreiben Sie ein Programm, das erkennt, ob ein String ein Palindrom ist.

(j) Schreiben Sie ein Programm, dass aus einer Liste aus `number's` Arrays und einem aus `string's` einen macht, indem es wie beim Reißverschluss die Elemente aneinanderhängt. Beispiel: `[1, 42, 0]` und `["x", "b", "drei"] -> [1, "x", 42, "b", 0, "drei"]`. Überlegen Sie, welchen Typen der Ergebnis-Array haben muss.